GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models

Zhiyuan Ma
zhiyuanma0520@gmail.com

***Abstract*—Tree speculative decoding verifies multiple candidate continuations in one target forward pass. For attention-only transformers, the verifier mainly needs an ancestry mask. Recurrent-hybrid language models break this assumption: a candidate row must also carry the recurrent state that native sequential decode would have produced along its root-to-node path. Otherwise, a verifier can use a correct attention mask while still conditioning on an impossible recurrent history.**

We present GDN Tree-Scan, a served verifier for Gated-DeltaNet hybrid language models integrated into vLLM. The system combines FlashAttention-2 tree-bias attention, branch-local GDN scan/replay, device-side multidraft commitment, and accepted-chain-only state publication. On the public Qwen3.6-27B-FP8 checkpoint, in a clean batch-one (B=1) SWE/Codex decode gate at temperature 0.6, a six-node root-branch tree reaches 23.88 token-weighted decode tokens/s versus 18.80 for native five-step MTP (E5), a 27.0% decode-throughput gain. The per-request-equal latency view is +4.0%, and end-to-end task wall time remains prefill-heavy. Empirical equivalence evidence is scoped to recurrent-oracle probability-rescore (p-rescore) closure within the observed native flip floor, not a full distribution-distance proof.

***Index Terms*—speculative decoding, recurrent models, Gated DeltaNet, vLLM, FlashAttention, ML systems**

I. Introduction

Speculative decoding accelerates autoregressive generation by drafting candidate tokens and asking the target model to verify them in parallel. Tree-shaped speculative decoding generalizes a single draft chain into a candidate token tree, improving the opportunity set for each target forward pass while preserving the target distribution under the verifier acceptance rule [1]–[4]. For attention-only Transformer stacks, the central verifier problem is largely an ancestry problem: row i may attend to the prompt and its ancestors, but not to sibling branches [1], [3].

Recurrent hybrid models break that simplification. Qwen3.6-style hybrid stacks contain many Gated-DeltaNet (GDN) recurrent layers in addition to softmax-attention layers, so a valid tree row needs two path-local facts: attention ancestry and recurrent state lineage [5], [6]. We use recurrent for any layer whose next-token output depends on a carried serving state; GDN is linear attention with such a carried state. The verifier must ensure that a node's GDN state equals the state native sequential decode would have produced along that node's root-to-node path. If sibling rows share a packed recurrent accumulator, a correct attention mask can still verify tokens from an impossible recurrent history. Figure 1 shows the minimal contamination case.

This paper reports a served verifier for a recurrent hybrid target inside vLLM. The contribution is not "tree speculative decoding" in the abstract. It is served-realization equivalence as an implementation target: forked FlashAttention-2 (FA2) tree bias for attention, branch-local GDN scan and accepted-chain replay for recurrence, and vLLM state publication that makes rejected branch state non-durable [7]–[9]. We are not aware of a prior public served implementation that closes this vLLM/FA2/GDN target-verifier boundary; the paper does not claim first hybrid speculation or first stateful tree decoding.

The measured result is deliberately narrow. We use E5 for the native five-step multi-token prediction (MTP) spine and catN for N-node tree variants. In a clean B=1, temperature-0.6 deployment trace over four SWE/Codex tasks, cat6root, a six-node tree with a depth-5 native spine plus one root sibling, reaches 23.88 token-weighted decode tokens/s versus 18.80 for native E5, a 27.0% decode-throughput gain. The root sibling is a depth-0 alternative: when native MTP would lose an event because the first spine token is rejected, the verifier can still commit the sibling branch, raising committed tokens/event from 4.11 to 4.82 while adding almost no verify-forward time. The per-request-equal latency view is 18.51 versus 17.80 tokens/s, or +4.0%; this is useful but underweights long decode turns where most generated tokens occur. Both cat6root and cat9 close within the observed native recurrent-oracle flip floor in a 40-turn B=1 p-rescore, but full distribution-distance evidence, request-level bootstrap, more seeds, B=4, and Stage D timing remain open gates.

The paper makes three concrete contributions:

- A served verifier contract for recurrent hybrid token trees: attention ancestry, path-local GDN state, SpecInfer-style residual commitment, and accepted-chain-only durable publication.
- A vLLM/FA2/GDN implementation path that preserves native serving machinery where possible and adds tree-specific state only at explicit seams.
- An evidence ledger that separates clean B=1 decode throughput, per-request latency, recurrent-oracle p-rescore, end-to-end wall caveats, and contaminated or still-open measurements.

II. Claim Scope and Evidence Map

We intentionally separate three claims: the correctness contract, clean B=1 decode throughput, and the empirical equivalence gate. Table I makes that boundary explicit. The public repository artifact preserves the longer development

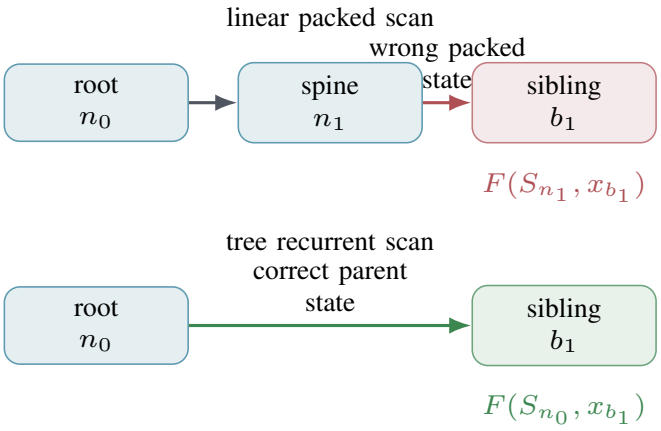

Fig. 1. A correct attention mask does not make a recurrent tree verifier correct. A sibling row packed after a spine row must inherit the root state, not the previous packed row's recurrent state.

record; the paper body carries the facts needed to evaluate the argument without following every repository link.

III. Background and Related Work

Speculative decoding uses a cheaper draft path to propose future tokens and verifies them with the target model. SpecInfer introduced tree-based speculative inference and parallel verification for LLM serving, while Sequoia, EAGLE-2, and OPT-Tree study hardware-aware, dynamic, or adaptive draft-tree structure [1]–[4]. These systems establish the candidate-tree and verifier-acceptance framing, but the usual target-forward assumption is too high-level for a served recurrent hybrid.

STree is the closest prior work for stateful tree decoding. It extends tree speculative decoding to SSM and hybrid architectures by exploiting accumulated state-transition structure [10]. That framing transfers directly: stateful models need tree-aware state construction, not only tree-aware attention. The missing layer for this artifact is realization. The served target path has fp8/bf16 seams, branch co-residency, a specific FA2 attention implementation, GDN state writeback, and rejection sampling inside vLLM.

Recent hybrid serving and SSM speculation work sharpens the same boundary. SpecMamba identifies hidden-state backtracking, tree-verification incompatibility, and draft/target workload mismatch for Mamba speculative decoding, then targets an FPGA dataflow [11]. Component-aware self-speculation shows that hybrid component structure matters: Falcon-H1 and Qwen3.5 behave differently because of how SSM/linear-attention components are integrated [12]. These papers explain why hybrid models create new verifier questions, but they do not close the vLLM/FA2/GDN served-realization path.

SGLang is the closest adjacent production comparison and should be treated carefully. Its hybrid-model writeup separates KV cache from request-level Mamba state, introduces MambaRadixCache, and handles speculative verification with

Algorithm 1 Tree verification on a recurrent hybrid target

Require: prefix state S_0, candidate tree $T = (V, E)$, token ids x_i, sampling state

Pack candidate ids, parent indices, and ancestry masks into vLLM decode metadata.
Run softmax-attention layers with FA2 tree bias so row i sees prefix and ancestors only.
for each GDN layer **do**
  for each node i in topological order **do**
    Load $S_{\text{parent}(i)}$ from S_0 or branch-local `h_cache`.
    Compute $S_i, o_i \leftarrow \text{GDNStep}(S_{\text{parent}(i)}, x_i)$.
    Store S_i in `h_cache[i]` for children.
  end for
end for
Apply the residual speculative acceptance rule on device.
Replay accepted path in native order and publish only accepted GDN/conv state.
return committed tokens and durable recurrent state for the next decode event.

independent state slots and parent tracing [13]. Its Qwen3-Next documentation and speculative-decoding documentation show active support for GDN/MTP-style hybrid models [14], [15]. That is not an apple-to-apple fused-kernel baseline for this paper: much of the mechanism is cache/scheduler-layer work, while this artifact's shipped contribution is lower in the stack, at the vLLM, forked-FA2, branch-local GDN scan/replay, and publication boundary [9], [16].

IV. Verifier Contract

GDN Tree-Scan verifies an arbitrary candidate tree, not only the native MTP drafter used in the current implementation. The input contract is a prefix state, a parent array, candidate token ids, and sampling state. The output contract is stronger: after verification and commitment, the next served decode event must start from the same observable token prefix and recurrent state that native sequential decode would have reached on the accepted tokens, up to the observed native recurrent-oracle numerical floor.

The contract can be summarized as an engineering invariant. Assume the tree contains only candidate ids and parent links; attention rows see only prefix plus ancestors; every GDN row is computed from its parent row's recurrent state; the committer applies the same residual rule as the target verifier; and durable state is replayed or published only for the accepted chain. Then the next served decode event has the same intended token-prefix and recurrent-state boundary as native sequential decode on the accepted prefix. The current evidence checks this boundary through recurrent-oracle p-rescore and targeted scan/replay gates rather than through a full distribution-distance proof [1].

V. System Design

Figure 2 shows the served path. The drafter currently uses Qwen's native MTP head: root and spine tokens come from

TABLE I

CLAIM, GATE, AND CAVEAT MAP. THE REPOSITORY HAS A LONGER RECEIPT LEDGER; THIS TABLE KEEPS THE ACADEMIC CLAIM BOUNDARY EXPLICIT.

Claim	In-paper evidence	Gate	Status and limit
The systems contribution is served-realization equivalence for Qwen GDN in vLLM.	FA2 tree-bias decode, branch-local GDN scan/replay, device multidraft commitment, and accepted-chain-only durable publication.	All target-verifier surfaces consume the same tree descriptor and accepted path.	Closed for the shipped B=1 path; not a claim of first GDN speculation.
cat6root is faster than native E5 on clean B=1 decode throughput.	Table VII: 23.88 versus 18.80 token-weighted decode tokens/s.	Matched pure-decode GPU timer, probes off, temp-0.6, four SWE/Codex tasks.	+27.0% decode-only; per-request-equal latency is +4.0%.
cat9 and cat6root close within the native recurrent-oracle floor.	Section VIII: 13.28% and 13.09% clear-margin flips versus native E5 at 12.90%.	Each arm is compared with its own no-spec recurrent oracle in the B=1 p-rescore.	Closed for this 40-turn sample; no TV-distance or request-cluster bootstrap yet.
The measurement discipline is part of the result.	Speed runs disable probes; equivalence runs enable oracle capture; token-weighted and per-request views are reported separately.	No mixing of raw prompt probes, oracle capture, B=4 co-residency, or denominator conventions.	Prevents false speed wins and false equivalence failures; B=4 remains open.

TABLE II

COMPARISON LAYER FOR PRIOR AND ADJACENT WORK. THE GOAL IS TO AVOID PROMOTING ADJACENT SERVING WORK INTO A DIRECT FUSED-KERNEL BASELINE.

Line of work	What transfers	What breaks for this target	What GDN Tree-Scan adds
SpecInfer, Sequoia, EAGLE-2, OPT-Tree	Candidate trees, verifier acceptance, dynamic or hardware-aware shape selection [1]–[4].	The target verification forward is assumed to realize the target distribution once attention ancestry is correct.	Recurrent-state verification and deployment equivalence gates.
STree	Stateful models need tree-aware state construction [10].	Algebraic state sharing is not the same as matching native served GDN under bf16/fp8 and vLLM writeback.	A GDN scan/replay kernel gated against native/deployment oracles.
SpecMamba	Hidden-state backtracking and tree verification are first-class SSM obstacles [11].	FPGA dataflow and FIFO verification are not a served GPU/vLLM path for Qwen GDN.	Fused GPU kernels, FA2/vLLM compatibility, and deployment gates.
Component-aware self-speculation	Hybrid component structure determines speculative viability [12].	Research Python implementation leaves fused CUDA, batched verification, and cache sharing as systems gaps.	A fused served path for Qwen GDN tree verification.
SGLang hybrid serving	Adjacent production mechanisms: dual memory pools, MambaRadixCache, parent tracing, MTP/EAGLE surfaces [13]–[15].	Cache/scheduler-layer mechanisms are not directly comparable to this fused vLLM GDN tree-scan kernel.	A different layer of realization: vLLM + forked FA2 tree bias + branch-local GDN scan/replay + observed recurrent-oracle gates.

TABLE III

ENGINEERING FORM OF THE ACCEPTED-STATE VERIFIER CONTRACT.

Part	Formal content	Served implementation
Inputs	Prefix state, parent array, candidate ids, sampling state.	vLLM tree packing plus multidraft metadata.
Attention	Row i sees prompt tokens and ancestors of i, not siblings.	Forked FA2 adds `tree_bias` before softmax; prefill stays native.
GDN	$S_i = \mathrm{GDNStep}(S_{\mathrm{parent}(i)}, x_i)$.	Triton GDN tree scan with branch-local `h_cache`.
Acceptance	Apply the same residual speculative rule as target verification.	Device multidraft commit at temperature 0.6.
Publication	Only accepted-prefix tokens and their state become durable.	Accepted-chain replay and vLLM state writeback/remap.

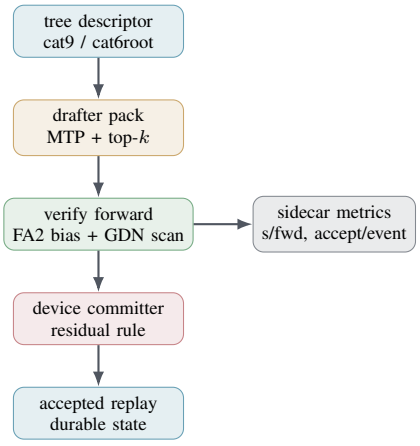

Fig. 2. The tree shape is a cross-layer contract. Candidate packing, attention metadata, recurrent scan, commitment, replay, and metrics must consume the same descriptor.

the same logits tensor, with runner-up leaves read as top-k candidates. The verifier should not depend on that drafter. Its stable boundary is a tree descriptor with parent indices, candidate ids, visibility masks, and accepted-path rows that can be replayed into native state.

A. FA2 Tree-Fork

The attention side needed a real FA2 fork. vLLM's generic tree attention path could express an additive ancestry bias through a Triton `unified_attention` route, but that route was not the same realization as native FA2 decode. The shipped fork leaves stock `varlen_fwd` unchanged and adds a separate `varlen_fwd_tree_bias` entry point for decode. For speculative query row i and speculative key row

j, the fork implements

$$\ell_{ij} = \text{scale}\, q_i k_j^\top + b_{ij}, \qquad b_{ij} = \begin{cases} 0, & j \in \text{ancestors}(i), \\ -\infty, & \text{otherwise}, \end{cases} \tag{1}$$

with prefix-context columns still visible to every row. The dense fp32 `tree_bias` matrix is passed as $[q, q]$ or batched $[B, q, q]$ metadata and is added after the QK score tile and before FA2 masking and softmax [7]. Internally the patch maps tile coordinates back to speculative suffix coordinates using $\text{context\_len} = \text{seqlen}_k - \text{seqlen}_q$ plus explicit query/key offsets, so the bias never applies to prefix columns by accident. The value injected into the FA2 accumulator is scaled so the external semantics are an additive bias on the scaled logits above.

The fork is decode-only by construction. Prefill remains on native FA2, and decode uses the fork only when `FR13_FA2_TREE_BIAS=1`, the decode metadata carries a non-empty bias matrix, and `max_query_len>1`. Empty or disabled tree metadata falls back to the existing vLLM attention path. ALiBi is fail-loud in the tree-bias route because adding two independent positional bias mechanisms would make the served-realization claim ambiguous.

The recurrent side needed branch-local state and publication discipline. During verification, each candidate row computes a GDN post-state from its parent row, not from the previous packed row. During commitment, the accepted chain is replayed in native order before durable state is written back. Rejected rows can influence acceptance decisions, but they must not become future model memory [16].

The descriptor passed through vLLM is intentionally drafter-neutral: parent indices, candidate ids, strict and visible ancestry masks, and accepted-path row ids. The current drafter is Qwen native MTP: the spine comes from the native MTP logits tensor, and runner-up leaves are top-k reads from the same candidate distribution. The verifier, however, only assumes that those rows have stable parent indices and acceptance probabilities. A suffix drafter, hybrid drafter, or future learned tree can reuse the same verifier if it supplies the same surfaces.

B. MTP Draft Head and Committer

The shipped drafter does not introduce an auxiliary model. It runs the native causal MTP spine and reads the LM-head logits once per draft step. The argmax of that tensor is the child-rank-0 spine token and is the only token fed back into the next MTP hidden-state step. Off-spine tree nodes are read-only runner-up tokens from the same logits tensor: cat9 reads depth-local top-2 leaves on the interior spine steps, while cat6root reads only the root runner-up and then keeps the rest of the native spine. This is why the root-sibling tree can add a depth-0 alternative without adding a second recurrent trajectory in the drafter. The single-logits drafter path is load-bearing here: it makes the spine argmax and the runner-up leaf come from the same full-vocabulary logits tensor instead of recomputing the LM head through a second sampling helper.

TABLE IV
SURFACES THAT MUST AGREE ON THE SAME TREE CONTRACT. MOST FAILED ROUTES BROKE ONE OF THESE ALIGNMENTS.

Surface	Required alignment	Typical failure if isolated
Candidate descriptor	Parent indices, candidate ids, visibility masks, and accepted-path rows match.	A drafter-specific shortcut cannot be replayed into native state.
FA2 attention	Decode rows see prefix plus ancestors, with prefill left on native FA2.	A correct abstract mask runs on a non-native attention realization.
GDN scan	Each node loads its parent recurrent state, not the previous packed row.	Sibling rows inherit impossible recurrent histories.
Committer	The residual speculative rule is applied to the same candidate rows verified by the target.	Accepted tokens are counted against stale or mismatched probabilities.
Publication	Only the accepted chain is replayed and made durable.	Rejected branch state leaks into the next decode event.

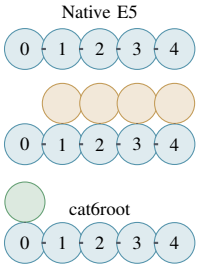

Fig. 3. The evaluated shapes. cat9 adds runner-up leaves to the depth-5 spine; cat6root keeps the spine and adds only a root sibling, reducing verifier rows while preserving a depth-0 alternative.

Partial tree acceptance also changes the next drafter input row. Stock linear MTP samples the next draft from the row at offset accepted_len. In a tree, the committed leaf may be a different flat verifier row. The tree path therefore maps the next sampled row to $\text{base} + \text{accepted\_path}[\text{len} - 1]$, or the root row when no draft token was accepted. This accepted-row sampling remap prevents the next MTP step from conditioning on a rejected sibling's hidden state after an alternative branch wins.

At temperature 0.6 the committed output is selected by the deployed device multidraft committer, not by a greedy longest-common-prefix shortcut. For each parent, it consumes the target distribution over the node's child draft tokens, computes the SpecInfer-style residual-mix source weights, samples a draft source, accepts with $\min(1, p/q_{\text{mix}})$, and falls back to the target residual otherwise. The default-on device multidraft path moves this per-node decision onto device, eliminating the host $[N_{\text{nodes}} \times |V|]$ softmax transfer and Python per-node loop while preserving the host reference distribution. The older pure-integer greedy GPU committer is only a temperature-0 tuning route; it is not the temperature-0.6 committer used for the reported deployment gate.

VI. GDN SCAN AND REPLAY KERNEL

The GDN recurrence is a stateful linear-attention update. For one value tile, the served kernel implements the same

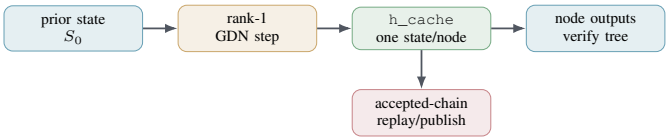

Fig. 4. The scan verifies all candidate nodes with parent-indexed recurrent state. Replay re-executes only the accepted chain and publishes durable GDN/conv state.

Algorithm 2 Branch-local GDN scan and publication

Require: Prior durable state S_0, tree parent array p, node features x_i, accepted path A
for node i in topological tree order **do**
  if p_i is root **then**
    $S_{\text{parent}} \leftarrow S_0$
  else
    $S_{\text{parent}} \leftarrow$ h_cache$[p_i]$
  end if
  $(o_i, S_i) \leftarrow \text{GDNNodeStep}(S_{\text{parent}}, x_i)$
  h_cache$[i] \leftarrow S_i$
end for
Device committer selects accepted node sequence A
$S \leftarrow S_0$
for node i in A in native order **do**
  $(o, S) \leftarrow \text{GDNNodeStep}(S, x_i)$
end for
Publish only replayed S and the matching convolution state

rank-1 update body used by the native path:

$$S'_t = \exp(g_t) S_{t-1}, \tag{2}$$

$$\Delta_t = \beta_t \left(v_t - S'_t k_t\right), \tag{3}$$

$$S_t = S'_t + \Delta_t k_t^\top, \tag{4}$$

$$o_t = S_t q_t. \tag{5}$$

For a tree node i, the invariant is $S_i = \text{native\_scan}(\text{tokens}(\text{root} \rightarrow i))$. This is simple to state and easy to violate in a packed GPU implementation [5]. A linearized tree scan is wrong whenever a branch follows a sibling in memory. If branch node b_1 is a child of root n_0, the required state is $S_{b_1} = F(S_{n_0}, x_{b_1})$, not $F(S_{n_1}, x_{b_1})$ for the previous packed spine node n_1.

The kernel keeps a post-state for each padded node in h_cache. That table has shape $[N_{\text{PAD}}, BV, DIM_K]$ in fp32, where BV is the value-channel tile width. This is the main local resource constraint and the reason the kernel has an explicit tree-size boundary rather than an unbounded tree descriptor.

A. BV, N_{PAD}, and Spill Boundary

We write BV16/w8 for a value tile of 16 rows launched with eight Triton warps. The shipped scan locks BV=16 and num_warps=8; the native packed-decode geometry audited from the pinned runtime is $BV = 32$, one warp, and three stages. N_{PAD} is the next power-of-two span used by the static tree masks and unrolled Triton loops. Thus a six-node cat6root tree occupies pad8, while nine-node cat9 and deeper deployed families occupy pad16; the runtime fails loud for activation rings with n_pad greater than 16 [9].

The resource model is simple enough to expose. With $DIM_K = 128$, the scan's cached post-state footprint is

$$M_h = N_{\text{PAD}} \cdot BV \cdot DIM_K \cdot 4 \text{ bytes}. \tag{6}$$

TABLE V
GDN SCAN GEOMETRY AND THE PAD16 RESOURCE BOUNDARY.

Geometry	h_cache	Resource fact	Role
Native one-node shape $BV32/w1/s3, N = 1$	16 KB	No full-tree state table.	Native packed-decode reference shape.
Full-tree native shape $BV32/w1/s3, N = 16$	256 KB	About 2048 cache fp32 values/lane before other live values; catastrophic spill.	Not deployable as a cached tree scan.
Deployed cat9 scan $BV16/w8, N = 16$	128 KB	Measured 254 regs/thread and 0 spill.	Shipped deepest tree-scan geometry.
cat6root scan $BV16/w8, N = 8$	64 KB	Half the cached state of cat9.	Explains near-native verify time with root-sibling gain.
Future pad32 $BV16/w8, N = 32$	256 KB	Register/SRAM hard wall for the cached scan.	Requires recompute or replay-style ancestry.

The corresponding register-pressure estimate for the cache alone is

$$R_h \approx \frac{N_{\text{PAD}} \cdot BV \cdot DIM_K}{32W}, \tag{7}$$

where W is the warp count. This estimate is not a substitute for ptxas, because the working tile and live q/k/v/g/beta values add pressure, but it predicts the failure mode: the full tree cache grows with padded node count, not with batch size.

This is why BV is not a free equivalence dial. Shrinking to $BV = 4$ reduces the cache, but it changes the tensor shape from native's $[32, 128]$ reduction to $[4, 128]$, changes compiler layout and reduction order, and increases the number of value-tile programs. The deployed $BV16/w8$ point is not chosen because it proves native SASS identity; it is chosen because it is output-bit-exact to the banked scan reference at $N_{\text{PAD}} = 1$ and 16, has measured zero spill, and preserves enough tile width for speed. The remaining bit-exactness obligation is against the native packed-decode geometry, which is why the code keeps an explicit diagnostic override and a separate recompute-from-spine route.

The spill boundary applies to the scan, not to publication. The accepted-path replay kernel carries one $[BV, DIM_K]$ state tile and re-executes only the committed chain, so its state footprint is $O(1)$ in tree size. The recompute-from-spine scan variant uses the same idea for verification: recompute each node's ancestry from the spine instead of caching every node post-state. That route can launch at native $BV32/w1/s3$, removes the pad16 state-cache wall, and is the right path for trees wider than the current deployed family. Its cost is extra rank-1 replay work, not local-memory spill on the saturated decode bandwidth path.

The scan and replay call the same node-step body. This is the key guard against a verifier that accepts under one recurrence but publishes under another. The body applies the native gate convention, beta's bf16 round trip, q/k normalization by division rather than reciprocal-square-root, output scaling, the rank-1 state update, and the carried-state bf16 boundary after output computation. These numeric seams are not cosmetic. In speculative decoding, a small branch-

TABLE VI
EXPERIMENTAL ASSUMPTIONS FOR THE REPORTED GATE.

Field	Value
Scope	B=1, temperature 0.6, four SWE/Codex tasks.
Target	Public `Qwen/Qwen3.6-27B-FP8`; local alias `qwen3.6-27b`.
Runtime	Pinned vLLM image, forked FA2 tree-bias decode path, and boot-time GDN patcher.
Primary metric	Token-weighted decode TPS.
Secondary metric	Per-request-equal TPS.
Not claimed	End-to-end task-wall gain, B=4 batching, or full TV-distance equivalence.

local numerical difference matters if it changes a clear-margin argmax or a residual sampling decision after many recurrent and attention layers.

The publication path also handles non-attention state. After the device committer chooses a path, the runtime remaps GDN state and convolution state using the accepted node rows, with gather-before-scatter ordering so overlapping source and destination columns cannot race. The convolution prior for the next event is taken from the accepted leaf before remap. This makes the durable state exactly the replayed native-order chain, while rejected rows remain temporary verifier state in `h_cache`.

VII. EXPERIMENTAL SETUP

The target checkpoint is the public Hugging Face model `Qwen/Qwen3.6-27B-FP8`, a 64-layer hybrid model whose documented layout has 48 GDN recurrent layers and 16 softmax-attention layers [6]. The serving environment mounts it at `/models/qwen3.6-27b-fp8` and exposes it as `qwen3.6-27b` on GB10. The source-of-truth runtime is a `vllm/vllm-openai` image pinned to digest `sha256:3dbe092ec5b2cef63b6104d33fa75d6ce53a7870962529ada69f78bbbc38e776`, corresponding to vLLM 0.19.2rc1.dev134+gfe9c3d6c5, plus the forked FA2 `.so` swap and the boot-time tree-GDN patcher. The clean B=1 tree arms use the forked FA2 tree-server launcher; cat9 also has a locked wrapper, native E5 uses the native speed launcher, and the B=1 deploy wrapper pins `GPU_UTIL=0.82`, `MAX_NUM_SEQS=1`, `MAX_MODEL_LEN=131072`, and `FR13_DEVICE_MULTIDRAFT=1`. The mounted model path and runtime image are pinned; the immutable Hugging Face snapshot hash inside the mount was not captured and is listed as a reproduction limitation. The workload is four SWE/Codex tasks in clean B=1 mode, temperature 0.6, with true sequential windows and `num_running` near zero.

Speed and empirical-equivalence evidence are measured in separate modes. Speed runs disable heavy capture/oracle probes; equivalence runs enable recurrent-oracle capture and p-rescore. This separation is not bookkeeping. Several earlier readings were confounded by raw prompt probes, B=4 co-residency, instrumentation overhead, or denominator mistakes, so the final result reports token-weighted decode throughput, per-request latency, p-rescore equivalence, and end-to-end wall caveats as different measurements.

A. Speed and Equivalence Infrastructure

The final gate uses the deployment regime, not the earlier raw-prompt microbench regime. The raw `/v1/completions` probes were useful for kernel debugging, but they produced off-distribution `<think></think>` loops and cross-boot trajectory forks that made accept/event non-comparable. The final runs use the SWE/Codex agent loop through the deployment proxy and bracket vLLM metrics per task. Engagement is fail-loud: native E5 must show five draft tokens per draft event, a tree arm must show $|T|$, and tree metadata must carry parent indices and accepted-path rows.

Speed mode is instrumentation-off except for low-overhead counters. It records raw vLLM deltas for decode seconds, draft events, accepted draft tokens, and generated tokens. When the pure-decode GPU timer is armed, `FR13_SFWD_GPU_TIMER` records asynchronous CUDA events around model forward only on uniform decode steps; it excludes prefill and mixed prefill+decode steps and drains completed events without per-step synchronization. Optional `FR13_DFWD_GPU_TIMER` and `FR13_CFWD_GPU_TIMER` sidecars attribute drafter and committer spans, but these attribution timers are not substitutes for the deployment-speed gate.

Equivalence mode is instrumentation-on and intentionally not used for speed. The proxy pair-dump is converted back to a served-token source stream, and `fr13_recurrent_decode_oracle.py` rescoring runs each arm's own no-spec recurrent decode path with speculative configuration disabled. This corrected an earlier oracle-frame mistake: chunked re-prefill is not the same as single-step recurrent decode for this hybrid target. The p-rescore then compares the served stream against the arm-local recurrent oracle and counts clear-margin argmax flips. That is why the paper reports a within-native-floor equivalence gate, not a bit-exact trace claim and not a speed number from an oracle-enabled boot.

Table VII is the core speed result. The accept/event column counts accepted draft tokens only. Each verify event also commits the verifier's guaranteed next token, so committed tokens/event is accept/event plus one: native E5 moves from 3.11 accepted drafts/event to 4.11 committed tokens/event, and cat6root moves from 3.82 to 4.82. This is a 17.2% committed-token gain while verify-forward time remains essentially native. The gain comes from the root sibling's position in the tree: it is a depth-0 alternative that recovers events native MTP loses when the first spine token is rejected, instead of spending most extra work on deeper leaves whose ancestors must all survive.

The three speed views are related but not interchangeable. Multiplying the committed-token gain by the pure verify-forward sidecar would predict a roughly 16–17% forward-only improvement, because $0.137/0.138$ s/fwd is nearly flat. The reported +27.0% is instead token-weighted deploy TPS: total

TABLE VII
CLEAN B=1 DECODE RESULT. THROUGHPUT DELTA IS TOKEN-WEIGHTED DECODE TPS RELATIVE TO NATIVE E5. ACCEPT/EVENT COUNTS ACCEPTED DRAFT TOKENS; COMMITTED TOKENS/EVENT EQUALS ACCEPT/EVENT PLUS ONE GUARANTEED VERIFIER TOKEN.

Arm	Tree	Per-request TPS	Token-weighted TPS	Delta	s/fwd_gpu	Accept/event
Native E5	Native 5-step MTP spine	17.80	18.80	baseline	0.137	3.11
cat9	9-node caterpillar: 5-spine + 4 leaves	18.44	22.63	+20.4%	0.144	3.64
cat6root	6-node root branch: 5-spine + root sibling	18.51	23.88	+27.0%	0.138	3.82

generated decode tokens divided by request decode seconds across the clean B=1 window. That denominator includes non-forward decode overhead and the weighting concentrates on long turns where most tokens are produced. The per-request-equal view gives +4.0% because it weights a short shell-tool call like a long edit turn. This paper reports all three numbers so the +27.0% headline is not read as a forward-only algebra or an end-to-end task-wall claim.

The request anatomy explains why these metrics differ. In the cat6root B=1 trace, 24.7% of turns have at most 64 output tokens, but those turns account for only 3% of output tokens; they are mostly short shell-tool calls in the Codex agent loop. Speculative decoding helps more on longer turns, so per-request-equal averaging undercounts the decode-token gain. This does not imply a 27% end-to-end task-wall gain, because the same workload repeatedly sends roughly 11k–14k prompt tokens and prefix caching is off for this hybrid setup. The final B=1 receipts do not promote a clean task-wall speedup; they establish that task wall improves by less than the decode-only gain unless deployment work reduces repeated prefill or overlaps streams.

VIII. EMPIRICAL EQUIVALENCE EVIDENCE AND MEASUREMENT DISCIPLINE

The empirical equivalence claim in this artifact is closure within the observed native recurrent-oracle flip floor. The exact B=1 depth-5 p-rescore compares each arm against its own no-spec recurrent oracle under temperature 0.6. Native E5 has 630 clear-margin flips over 4,883 positions, or 12.90%; cat9 has 645 over 4,858, or 13.28%; cat6root has about 636 over about 4,860, or 13.09%. Native E5 has a nonzero flip rate because the oracle is not a bit-identical replay of the served trace; it is a no-spec recurrent decode path, so native E5 establishes the serving-frame and numerical floor. Both tree arms are within the native E5 floor by less than one position-level standard error, but this is not a TV-distance estimate and not a request-cluster bootstrap.

The statistical caveat matters. The p-rescore sample has roughly 4.9k positions per arm, but positions inside one SWE/Codex turn are correlated. The result is therefore clean B=1 closure for the observed sample, not a universal guarantee across all seeds, batch sizes, or prompt distributions. Request-cluster bootstrap, additional seeds, and full distribution-distance measurements are follow-up gates.

The measurement infrastructure is part of the contribution because it prevented false wins and false losses. The canonical

TABLE VIII
EMPIRICAL EQUIVALENCE CLOSURE GATES AND INTERPRETATION.

Gate	Checked	Status
Depth-5 p-rescore	E5/cat9/cat6root temp-0.6 recurrent oracle.	Within native floor.
GDN scan A/B	$BV16 = BV32 = 0.0$ in controlled scan.	Scan alone is clean.
Native path oracle	Path-local state versus native decode order.	Core invariant.
Device committer	Same residual-mix rule on device.	Residual-rule gate.
Broad cat9 floor	Big-denom deployment consolidation.	Overlapping floor intervals.

formulas are:

$$\text{s/fwd\_gpu} = \frac{\text{decode forward GPU seconds}}{\text{pure decode drafts}}, \quad (8)$$

$$\text{accept/event} = \frac{\text{accepted draft tokens}}{\text{draft events}}, \quad (9)$$

$$\text{committed/event} = \text{accept/event} + 1, \quad (10)$$

$$\text{deploy TPS} = \frac{\text{request decode tokens}}{\text{request decode seconds}}. \quad (11)$$

Probe separation, deployment-path-only measurement, token weighting, and oracle-mode separation are all needed to make the numbers interpretable. This is why raw `/v1/completions` probes, B=4 screens, and oracle-enabled equivalence traces are not treated as substitutes for the clean B=1 deployment-speed gate.

IX. DESIGN EVOLUTION AND NEGATIVE RESULTS

The failure library explains why the final verifier has its particular contract. It separates three questions that can otherwise blur together: whether the verifier preserves the target distribution within the native floor, whether the kernel is fast enough to matter, and whether the measurement regime is timing the served path. Table IX condenses the HTML failure library into paper form.

Several rows are useful future work rather than dead ends. WY-style whole-tree algebra may still become a speed route if it can be tied back to the served sequential oracle. Cache-side or radix-style state sharing in vLLM would also be complementary to the current verifier rather than a replacement for it. The important point is that none of those routes is allowed to stand in for accepted-chain replay, path-local GDN state, or the deployment/equivalence separation used in the reported result.

TABLE IX
NEGATIVE RESULTS AND SUPERSEDED INTERPRETATIONS THAT SHAPED THE FINAL VERIFIER. THESE ARE NOT ALL MATHEMATICAL IMPOSSIBILITIES; SEVERAL ARE REJECTED ONLY AS SERVED-PROOF ROUTES FOR THIS ARTIFACT.

Route or interpretation	Why it looked plausible	What failed	Paper consequence
Attention-mask-only tree	Tree verification for attention-only LLMs mainly needs ancestry.	GDN rows still carried sibling-contaminated recurrent state.	Attention ancestry is necessary but not sufficient.
Shared packed recurrent accumulator	Co-resident rows are efficient if packed sequentially.	Packed order is not native branch order, so sibling states leak.	The scan must be parent-indexed and branch-local.
STree-style state sharing as direct transplant	STree gives the right stateful-tree problem lens.	Algebraic state construction is not the same as matching served bf16/fp8 GDN writeback.	STree is prior work, not the served-realization proof.
Triton `TREE_ATTN` as proof path	It can express a tree mask.	It is not the same FA2 decode realization as native serving.	The reported path uses a forked FA2 tree-bias decode kernel.
WY / whole-tree algebra	It could reduce state traffic and expose more parallelism.	Different summation trees and bf16 boundaries are not the native sequential verify oracle.	Kept as future speed work; not the equivalence proof route.
Literal 0.0 max-abs as only equivalence bar	Bit equality is easy to state.	Native recurrent-oracle comparisons already have a numerical flip floor.	The paper uses within-native-floor p-rescore, not bit-exact intermediate equality.
cat10 and broad B=4 screens	Wider trees and batching are deployment-relevant.	Contaminated reads violated the expectation that a strict tree superset should not accept fewer spine tokens than native.	Clean B=1 is the mechanism proof; B=4 needs a refreshed carrier-clean run.
Per-request TPS as headline	It is a latency-friendly summary.	It weights a short shell-tool turn like a long edit turn.	Token-weighted decode TPS is the throughput headline; per-request TPS is a caveat.

X. THREATS TO VALIDITY

The result is strong for clean B=1 decode throughput and within-floor closure under the current recurrent-oracle p-rescore. It does not claim that every deployment bottleneck is solved.

- **Equivalence sample size.** The exact cat9/cat6root p-rescore is a 40-turn B=1 sample per arm. The position-level standard error is informative but optimistic because positions within a turn are correlated; request-level bootstrap and more seeds remain open.
- **Decode is not full task wall.** The +27.0% number is decode throughput. SWE/Codex requests repeatedly send large repository context, and prefix caching is off for this hybrid setup, so end-to-end wall gains are smaller; the final receipts do not contain a clean promoted task-wall number.
- **B=4 needs refreshed treatment.** The clean mechanism result is B=1. Batching changes co-residency, scheduler pressure, and prefill/decode overlap, so the old B=4 diagnostics are not a final verdict.
- **Stage D timing is open.** Drafter, verify forward, and isolated committer compute are near E5, but live phase attribution still needs direct step-wall timers.
- **Model snapshot packaging.** The model name and local mount are pinned, but the immutable Hugging Face snapshot hash inside the mount was not captured in the current receipts.

XI. CANONICAL CONFIGURATION

The shipped cat9 and cat6root arms share one tree-serving configuration. The only intended per-arm difference is the speculative tree shape: cat9 is a 9-node depth-5 spine plus leaves, while cat6root is a 6-node depth-5 spine plus root sibling. Table X lists the load-bearing choices most likely to invalidate the result if changed.

TABLE X
LOAD-BEARING CONFIGURATION CHOICES IN THE SHIPPED TREE PATH.

Surface	Choice
Tree mode	`FR10_ENABLE_TREE_GDN=1`, `FR10_DECODE_MODE_DEFAULT=tree_mtp`.
Attention	`FR13_FA2_TREE_BIAS=1`, `FR13_FA2_PREFILL_NATIVE=1`.
GDN scan geometry	`BV=16`, `num_warps=8`, activation-ring `n_pad`$\leq$ 16; cat6root uses pad8 and cat9 uses pad16.
Drafter rows	`FR13_DRAFTER_SINGLE_LOGITS=1`, `FR13_TREE_SAMPLE_ROW=1`.
GDN batch variance fix	`LUMO_FB_KERNEL_ROWS=1`, `LUMO_FB_PROJ_PAD_ROWS=16`.
Committer	`FR13_DEVICE_MULTIDRAFT=1` default-on for tree arms.
Safety	`FR10_ALLOW_LINEAR_FALLBACK` unset.
Operational B=1	`GPU_UTIL=0.82`, `MAX_NUM_SEQS=1`.

The committer row follows the later bake addendum: the older canonical-config catalog still lists `FR13_DEVICE_MULTIDRAFT` as diagnostic/off, while the addendum records that it was baked default-on for tree arms after verifying that native E5 does not read the tree committer gate. Some flags remain explicit on purpose. The FA2 tree-bias and native-prefill flags are default-off in the patcher to preserve patch idempotency anchors; baking them would require a separate re-patch verification task. The fixed-row in-projection pad should not be baked globally because it can perturb native E5 bytes, which define the recurrent-oracle baseline.

XII. CONCLUSION

GDN Tree-Scan turns a token tree into a served verifier for a recurrent hybrid language model by treating recurrent state as part of the tree contract. Attention ancestry is necessary, but not sufficient. The shipped path adds FA2 tree bias for softmax

layers, branch-local GDN scan/replay for recurrent layers, and accepted-chain-only state publication for vLLM. Under the clean B=1 deployment gate, cat6root improves token-weighted decode throughput by 27.0% while closing within the observed native recurrent-oracle flip floor. The result should be read with its caveats attached: decode-only, B=1, 40-turn p-rescore, no request-cluster bootstrap, no full distribution-distance estimate, prefill-heavy workload, and open B=4/Stage D follow-up.

Code and Artifacts

The implementation and measurement package is available in the public repository artifact pinned at commit `55f55854328b37f262e97d57b5863d8fadd7ff76` [9]. It includes the forked FA2 tree-bias patch, Triton GDN scan/replay kernel, vLLM wiring patch, launchers, and measurement harness used for the reported B=1 run.

References

[1] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2024.

[2] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, robust, and hardware-aware speculative decoding," 2025.

[3] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-2: Faster inference of language models with dynamic draft trees," 2024.

[4] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "Opt-tree: Speculative decoding with adaptive draft tree structure," 2025.

[5] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving mamba2 with delta rule," 2025.

[6] Qwen Team, "Qwen3.6-27B-FP8." https://huggingface.co/Qwen/Qwen3.6-27B-FP8, 2026. Public Hugging Face model page; accessed 2026-06-18.

[7] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023.

[9] Z. Ma, "GDN Tree-Scan consolidated implementation and measurement receipts." https://github.com/MaCoredroid/Lumo_FlyWheel/tree/55f55854328b37f262e97d57b5863d8fadd7ff76, 2026. Commit-pinned repository receipts for deploy TPS, equivalence gates, runtime, launchers, kernels, and configuration.

[10] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state-space models," 2025.

[11] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "Specmamba: Accelerating mamba inference on fpga with speculative decoding," 2025.

[12] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026.

[13] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention." https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/, 2025.

[14] SGLang Project, "Qwen3-Next SGLang documentation." https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next, 2026.

[15] SGLang Project, "Speculative Decoding Documentation." https://sgl-project.github.io/advanced_features/speculative_decoding.html, 2026.

[16] SGLang Project, "Hybrid-GDN MTP speculative decoding is not lossless on Ascend NPU." https://github.com/sgl-project/sglang/issues/25587, 2026.